EXPERIMENT NO. 5

Name of Student	**Mahi Jodhani**
Class Roll No	**21**
D.O.P.	**04-03-25**
D.O.S.	**18-03-25**
Sign and Grade	

AIM:

To create a Flask application that demonstrates template rendering by dynamically generating HTML content using the render_template() function.

PROBLEM STATEMENT:

Develop a Flask application that includes:

1. A homepage route (/) displaying a welcome message with links to additional pages.
2. A dynamic route (/user/<username>) that renders an HTML template with a personalized greeting.
3. Use Jinja2 templating features, such as variables and control structures, to enhance the templates.

THEORY:

1. What does the render_template() function do in a Flask Application?

   The render_template() function in Flask is used to render HTML templates stored in the templates folder. Instead of returning plain text from a route, Flask can return a complete HTML page using this function.

   Key Features of render_template():

   - Loads an HTML file and returns it as the HTTP response.
   - Allows passing dynamic content (variables) to the template.
   - Supports template inheritance, which helps in maintaining reusable layouts.
   - Uses **Jinja2 templating** to enable dynamic content rendering.

2. What Is the Significance of the templates Folder in a Flask Project?

   The templates folder is a **default directory** in Flask where all HTML files (templates) are stored. Flask automatically looks for templates in this folder when using render_template().

   Why Is the templates Folder Important?

   - **Separation of Concerns:** It keeps HTML files separate from Python code, following the MVC (Model-View-Controller) architecture.
   - **Organized Structure:** Helps in managing multiple pages and layouts efficiently.
   - **Automatic Lookup:** Flask automatically searches for HTML templates inside this folder.
   - **Supports Template Inheritance:** Allows reusing layouts using a base template.

3. What Is Jinja2, and How Does It Integrate with Flask?

   Jinja2 is a powerful templating engine used in Flask to dynamically generate HTML content. It allows embedding Python-like expressions inside HTML templates.

Key Features of Jinja2 in Flask:

- Supports **template variables** ({{ variable }}) for dynamic content.
- Allows **control structures** like loops ({% for %}) and conditionals ({% if %}).
- Supports **template inheritance** for reusable layouts.
- Provides **filters** and **macros** to modify output.

How Jinja2 Integrates with Flask?

- Flask uses Jinja2 internally to process templates.
- When calling render_template(), Flask renders the template using Jinja2.
- Developers can pass Python variables, lists, and objects to templates dynamically.

Github Link:- https://github.com/mahijodhani/Webx-Exp5

CODE: -

App.py

```
from flask import Flask, render_template, request, redirect, url_for
app = Flask(__name__)
@app.route('/')
def home():
    return render_template("home.html")
@app.route('/profile/<username>')
def profile(username):
    return render_template("profile.html", username=username)
@app.route('/about')
def about():
    return render_template("about.html")
@app.route('/contact', methods=["GET", "POST"])
def contact():
    if request.method == "POST":
```

```
        name = request.form["name"]
        message = request.form["message"]
        return render_template("contact.html", submitted=True, name=name, message=message)
    return render_template("contact.html")
@app.route('/interests', methods=["GET", "POST"])
def interests():
    interests = None
    if request.method == "POST":
        interests = request.form["interests"]
    return render_template("interests.html", interests=interests)
if __name__ == '__main__':
    app.run(debug=True)
```

templates/home.html

```
{% extends 'base.html' %}
{% block title %}Home{% endblock %}
{% block content %}
<h1>Welcome to Mahi's Flask App</h1>
<p>This is the homepage. Use the navigation bar to explore more.</p>
{% endblock %}
```

templates/base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Flask App{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <nav>
        <a href="{{ url_for('home') }}">Home</a>
        <a href="{{ url_for('profile', username='Mahi') }}">View Profile</a>
        <a href="{{ url_for('about') }}">About Me</a>
        <a href="{{ url_for('contact') }}">Contact</a>
        <a href="{{ url_for('interests') }}">Interests</a>
    </nav>
```

```
    <div class="content">
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

templates/about.html

```
{% extends 'base.html' %}

{% block title %}About Me{% endblock %}

{% block content %}
<h2>About Mahi</h2>
<p>I am passionate about web development, AI, and crafting clean UI experiences.</p>
{% endblock %}
```

templates/contact.html

```
{% extends 'base.html' %}

{% block title %}Contact{% endblock %}

{% block content %}

<h2>Contact Me</h2>
<form method="POST">
    <label>Name:</label><br>
    <input type="text" name="name" required><br><br>
    <label>Message:</label><br>
    <textarea name="message" required></textarea><br><br>
    <button type="submit">Send</button>
</form>
{% if submitted %}
<p><strong>Thank you {{ name }}!</strong> Your message was: "{{ message }}"</p>
{% endif %}
{% endblock %}
```

templates/profile.html

```
{% extends 'base.html' %}

{% block title %}Profile{% endblock %}

{% block content %}
<h2>Hello, {{ username }}!</h2>
<p>This is your profile page.</p>
{% endblock %}
```

templates/interests.html

```
{% extends 'base.html' %}

{% block title %}Interests{% endblock %}
```

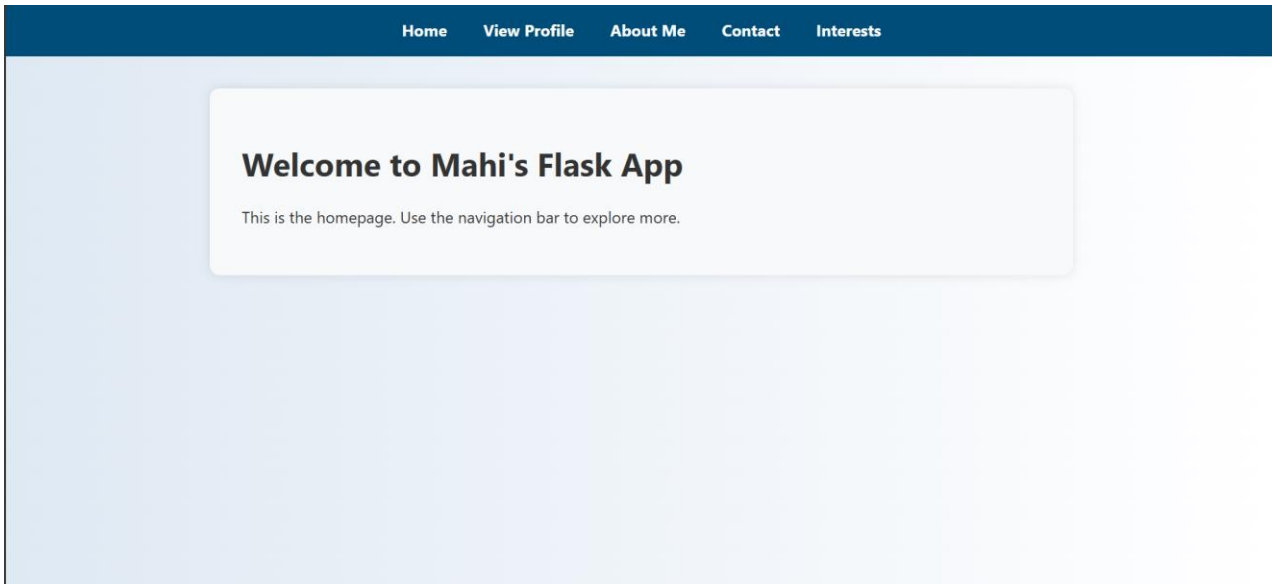
```
{% block content %}
<h2>My Interests</h2>
<form method="POST">
    <label for="interests">Type your interests:</label>
    <input type="text" name="interests" required>
    <button type="submit">Submit</button>
</form>

{% if interests %}
<p><strong>You entered:</strong> {{ interests }}</p>
{% endif %}
{% endblock %}
```

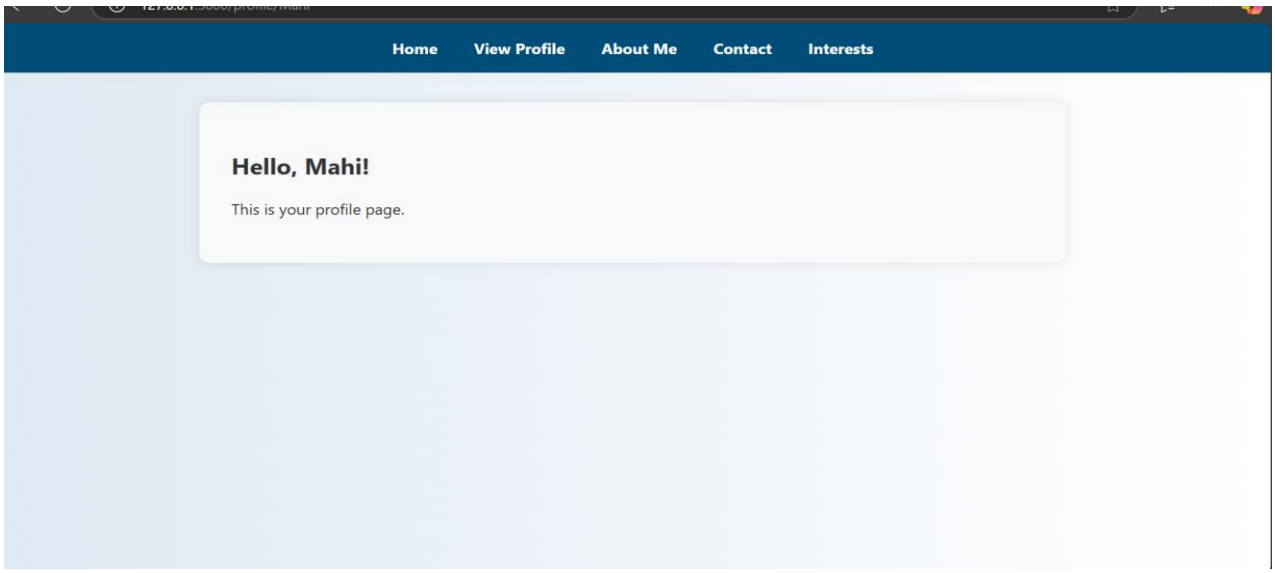
static/style.css

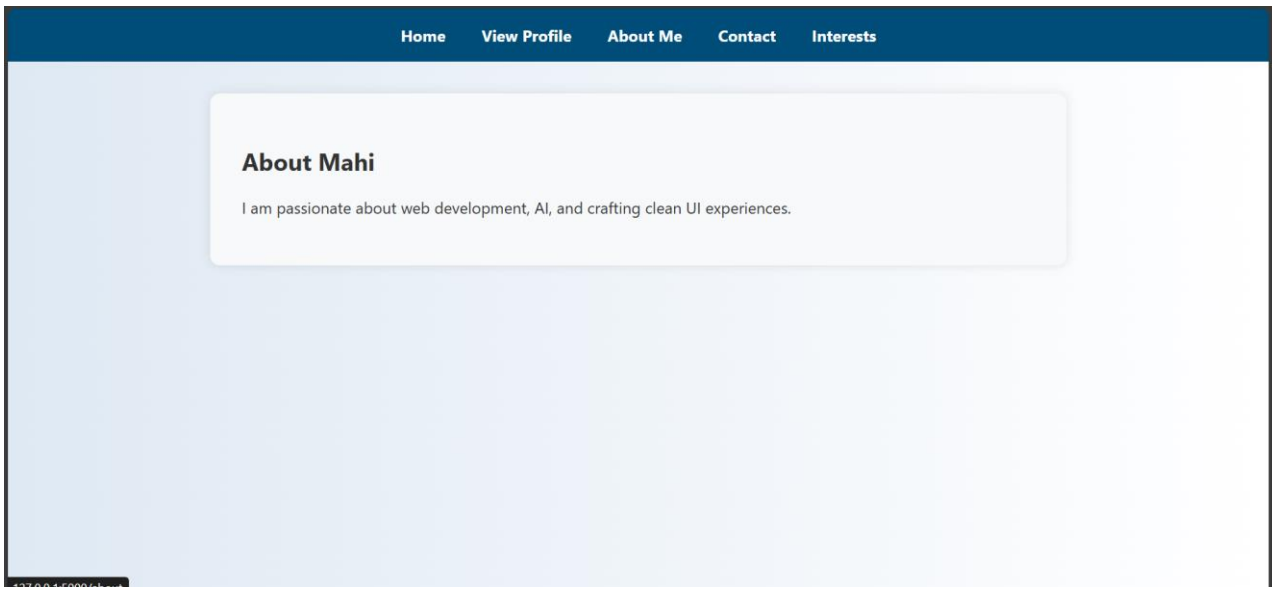
```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(to right, #dfe9f3, #ffffff);
    color: #333;
    margin: 0;
    padding: 0;
}

nav {
    background-color: #004d7a;
    padding: 1em;
    text-align: center;
}

nav a {
    color: #fff;
    margin: 0 1em;
    text-decoration: none;
    font-weight: bold;
}

nav a:hover {
    text-decoration: underline;
}

.content {
    max-width: 800px;
    margin: 2em auto;
    padding: 2em;
    background-color: #f7f9fb;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    border-radius: 10px;
}

input[type="text"], textarea {
    width: 100%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    margin-bottom: 1em;
```

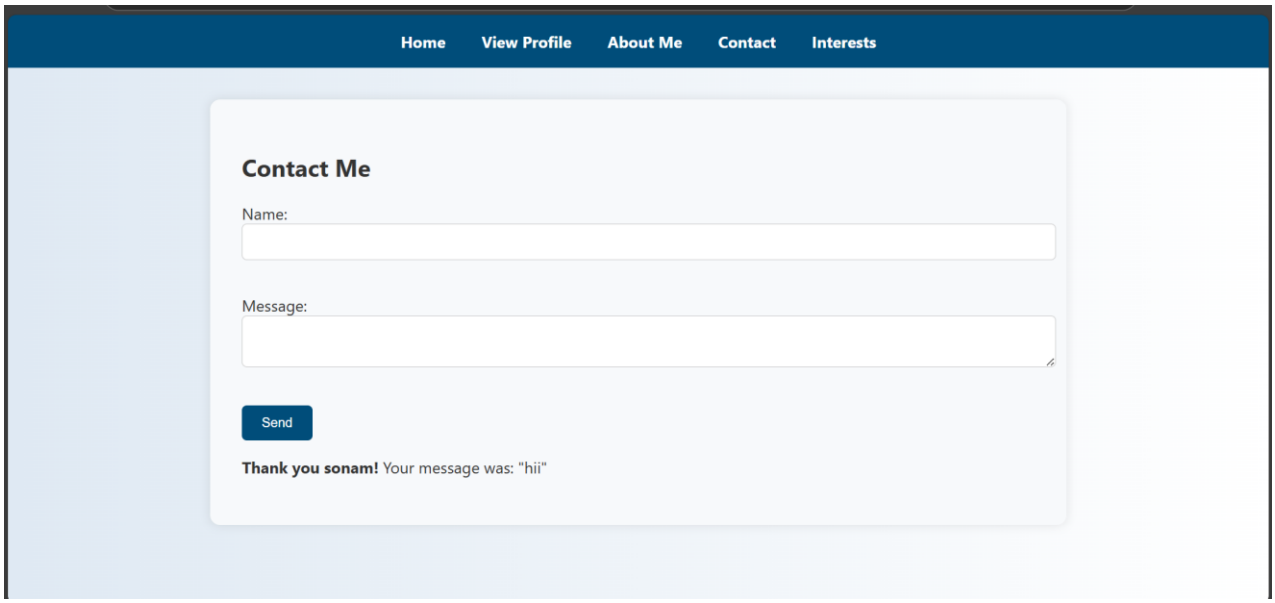
```
}

button {
    background-color: #004d7a;
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #003954;
}
```

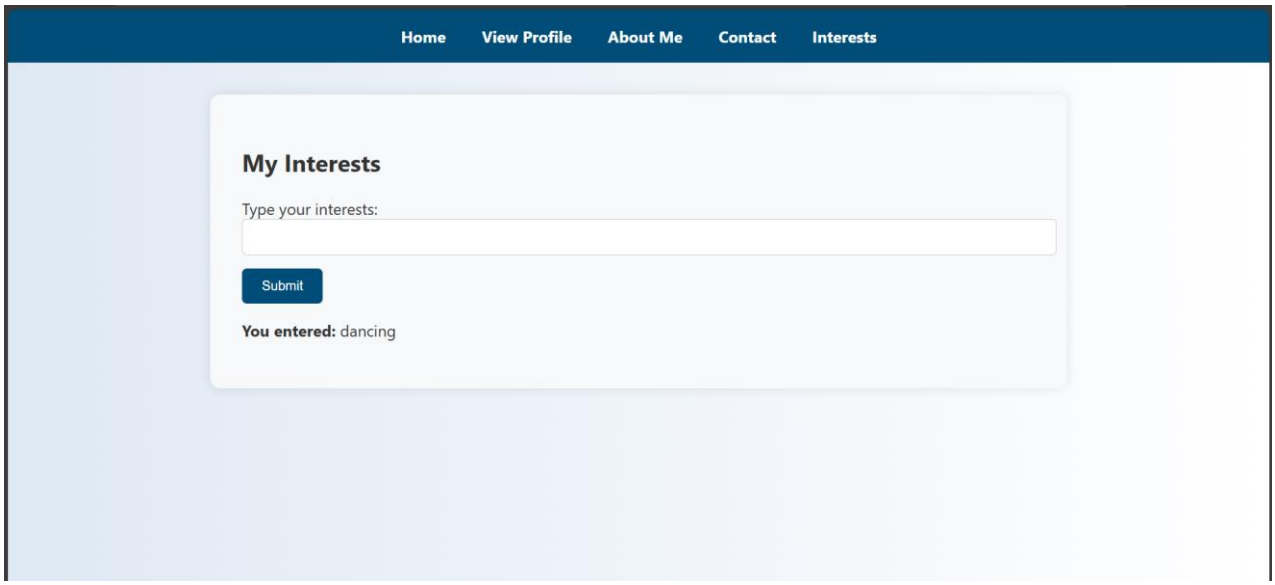
OUTPUT: -






Home
View Profile
About Me
Contact
Interests
About Mahi
I am passionate about web development, AI, and crafting clean UI experiences.


Home
View Profile
About Me
Contact
Interests
Contact Me
Name:
Message:
Send
Thank you sonam! Your message was: "hii"



Conclusion : -

The Flask application successfully demonstrates template rendering using the render_template() function and Jinja2 templating features. The homepage provides navigation links, while the dynamic user route personalizes content by passing data to an HTML template. By utilizing variables and control structures, the application efficiently generates dynamic web pages, showcasing Flask's ability to create flexible and interactive web applications.